

Delimitação de originalidade

Fábrica de software multi-agente — a proposta, sua origem e sua posição no estado da arte

Enzo Consulo · 29 de agosto de 2026 · repositório público: github.com/enzoconsulo/Generic-Multi-Agents

Este documento responde a uma pergunta feita em orientação: já existindo fábricas de software agênticas, o que este trabalho acrescenta. A resposta exige duas coisas que uma tabela de funcionalidades não dá — dizer **de onde a proposta veio** e dizer **em que regime ela opera**. Sem a segunda, a comparação engana: sistemas que resolvem problemas de tamanhos diferentes parecem concorrentes quando não são.

1. O regime: por que a comparação precisa começar aqui

O artigo do ChatDev reporta o consumo médio de **22.949 tokens para produzir um software inteiro** (MetaGPT, 29.279; GPT-Engineer, 7.183). Neste trabalho, o contexto de **um único despacho** ficou entre 11 e 14 mil tokens depois de otimizado — e um trabalho medido em operação real consumiu 2,44 milhões de tokens só de leitura. São duas ordens de grandeza de diferença, e elas não são detalhe: são o que define quais problemas existem.

	ChatDev / MetaGPT	softwarefabrik.io	Este trabalho
Unidade de trabalho	Um programa, numa execução	Um projeto, com aprovação humana a cada passo	Um projeto, decomposto em 8 a 20 tarefas
Duração típica	Uma sessão	Uma sessão assistida	Semanas, com sessões independentes
Custo por unidade	≈ 23 mil tokens por software	Estimado antes da execução	139 execuções medidas; até 2,44 M de leitura numa
Escala observada	Programas de avaliação	Projetos com operador presente	72 tarefas num só projeto, 86 concluídas em três

A CONSEQUÊNCIA Num regime de sessão única, perda de contexto quase não morde: tudo cabe no diálogo. Num regime de semanas, ela é o problema central — e passam a ser necessários mecanismos que não fazem sentido no primeiro: estado que sobrevive à sessão, contexto diferenciado por papel, prefixo estável para reaproveitamento e contabilidade por chamada. Comparar os sistemas sem dizer isso faz parecer que uns têm recursos que outros não têm, quando na verdade resolvem problemas diferentes.

2. A origem da proposta

A proposta não nasceu de um produto observado, e sim de um diagnóstico registrado na documentação do trabalho antes de existir código: quando se pede a um modelo de linguagem que construa um sistema inteiro, **as falhas não são de escrita, são de processo**. O modelo continua produzindo bem cada trecho isolado; o que se perde é a coerência do conjunto. Quatro modos de falha aparecem sempre, e nenhum se resolve com um modelo melhor: perda de contexto, decisão sem rastro, autoaprovação e tentativa sem limite.

Daí saem as três regras que a documentação apresenta como origem de todo o resto — quem implementa nunca é quem aprova; estado fora da conversa; falha limitada — e a tese: o gargalo não está na geração de texto, e sim na governança. A segunda versão do sistema leva a tese ao limite: se a governança é o gargalo, governança escrita em prosa dentro de um prompt é governança opcional, e o que puder ser propriedade da estrutura não deve ser frase.

3. As quatro dimensões em que os sistemas realmente diferem

A pergunta útil não é quem tem portão de qualidade — todos têm. São estas quatro, e cada uma é verificável na documentação dos respectivos projetos.

3.1 Onde vive o estado do trabalho

É a dimensão que decide todas as outras, porque define o que sobrevive ao fim de uma execução.

Sistema	Onde o estado vive	O que acontece ao fim da sessão
ChatDev / MetaGPT	Memória de diálogo, em dois níveis: curta dentro da fase, longa entre fases, compartilhando apenas as soluções de cada sub tarefa	A execução termina; não há continuidade prevista entre execuções
softwarefabrik.io	Espaço de trabalho versionado, mais o operador humano, que sustenta a continuidade	O operador retoma; o estado do processo é a aprovação pendente
Este trabalho	Arquivo de tarefa com estado explícito, e banco transacional na segunda versão; a conversa nunca é fonte	Nada se perde: a próxima execução reconstrói lendo o estado e o histórico de versões

O ChatDev **comprime** o histórico entre fases, o que é uma resposta real à perda de contexto dentro do regime dele. O que ele não faz — porque não precisa — é sobreviver ao encerramento. Aqui, um agente começa frio a cada despacho **por desenho**, e é o sistema que precisa lhe entregar o que ele precisa saber.

3.2 O que cada papel recebe

Esta é a dimensão mais específica deste trabalho, e a que menos aparece nos demais. O impulso natural é dar a cada agente o máximo de informação possível; aqui isso é prejuízo, e o motivo é o laço: **um arquivo que o revisor nunca vai abrir não é um desperdício de uma vez, é um desperdício multiplicado pelo número de voltas daquele despacho**.

Sistema	O que o construtor recebe	O que o revisor recebe
ChatDev / MetaGPT	O histórico da fase e as soluções das fases anteriores	O mesmo histórico da fase; o papel muda, o contexto não
softwarefabrik.io	O espaço de trabalho montado no contêiner	As diferenças finais, para decisão humana ao fim da execução
Este trabalho	O índice denso do projeto e os arquivos que a tarefa declara tocar	As mudanças e nenhum fonte inteiro; o verificador recebe os critérios e o resultado da passada mecânica já pronto

A formulação que resume o mecanismo está na documentação: **a informação que sobra em um papel é justamente a que falta em outro**. Medido na primeira versão, o revisor passou a sair com nenhum arquivo embutido, contra dois ou três do construtor da mesma tarefa, e o contexto do papel caiu de cerca de 35 mil para 15 mil tokens.

3.3 Como o custo é tratado

Sistema	O que mede	Para quê
ChatDev / MetaGPT	Consumo agregado por tarefa, publicado no artigo	Comparar frameworks em avaliação
softwarefabrik.io	Estimativa local em tokens e euros, antes da execução, sem chamar o fornecedor	Informar o operador antes de gastar
Este trabalho	Contabilidade por chamada, com separação entre entrada a preço cheio, escrita e leitura de memória temporária do fornecedor	Reprojetar o sistema: a medição decide onde mexer

A diferença não é de precisão, é de finalidade. Medir para comparar e medir para estimar são úteis; medir **para reprojetar** é o que produz achado. Foi assim que se descobriu que 93,14% dos tokens de entrada já eram leitura reaproveitada, e que a escrita — com 6,66% dos tokens — carregava cerca de metade da conta. O alvo do redesenho mudou por causa desse número.

3.4 Quem aprova, e o que acontece quando falha

Aqui é preciso corrigir uma simplificação comum: os três sistemas separam quem escreve de quem aprova, e dois deles limitam a repetição. A diferença está em como.

Sistema	Quem aprova	O que acontece ao falhar
ChatDev / MetaGPT	Agente revisor, no mesmo diálogo; ele aponta e o programador corrige — não edita o código	Limite declarado: duas modificações sem mudança, ou dez rodadas de comunicação
softwarefabrik.io	O operador humano, com as diferenças à vista, num estado de espera por aprovação	O operador decide: aprova, rejeita ou intervém
Este trabalho	Dois agentes, com perguntas distintas — funciona? e é o que foi pedido? — e a ferramenta de escrever não existe para eles	Escada de quatro degraus: sobe de modelo, troca de especialista, replaneja a tarefa em menores, e só então bloqueia

Três observações honestas. A primeira: o revisor do ChatDev também não edita código, e o limite de dez rodadas é um limite real — versões anteriores deste documento afirmavam o contrário, e estavam erradas. A segunda: o que distingue aqui não é ter limite, e sim a escada — em particular o terceiro degrau, que trata o fracasso repetido como possível erro de dimensionamento da tarefa e a redivide automaticamente. A terceira: a proibição de corrigir, aqui, não é convenção do protocolo de diálogo; é ausência da ferramenta no catálogo do papel, com teste que percorre os papéis para garantir.

4. A economia do contexto: as cinco alavancas

Esta seção é a mais concreta do trabalho e a que menos tem equivalente publicado. Ela parte de três fatos encadeados: a interface do modelo não tem memória, então todo o histórico é reenviado a cada chamada; logo o texto de abertura é cobrado de novo a cada volta; e o fornecedor permite reaproveitar esse começo por cerca de um décimo do preço, desde que ele seja **idêntico caractere por caractere**.

Alavanca	O que é	Efeito
Começo estável e reaproveitado	Ferramentas, doutrina e índice do projeto idênticos entre despachos	O trecho repetido cai para cerca de um décimo do preço
Contexto por papel	Quem constrói leva os arquivos da tarefa; quem revisa leva só as mudanças	Contexto não usado é pior que ausente: é relido a cada volta
Chechagem por comando antes do modelo	Critérios de aceite escritos como comando executável rodam de graça, antes do despacho	O que falha volta sem pagar um agente para confirmar o óbvio
Modelo conforme o papel	Verificar é mecânico e usa o modelo barato; o retrabalho sobe de modelo	A diferença entre as faixas é de cinco vezes no preço de entrada
Trabalho em lote	Reindexação e documentação não precisam de resposta imediata	A via assíncrona do fornecedor cobra metade pelo mesmo trabalho

A ARMADILHA QUE ANULA TUDO, EM SILÊNCIO Uma data, um contador ou um número de versão dentro do trecho estável invalida tudo o que vem depois — sem erro e sem aviso, apenas com a economia que não acontece. Por isso o sistema traz um teste que monta esse começo duas vezes, em momentos diferentes, e falha se os bytes divergirem. É um caso exemplar da tese do trabalho: a regra não é pedida ao modelo, é garantida por um teste.

A segunda alavanca merece o destaque porque contraria a intuição, e porque é onde a diferença em relação aos demais é mais clara: nos sistemas comparados, o papel muda e o contexto não. **Aqui o contexto é função do papel**, e essa decisão está isolada num único módulo, para que a política seja auditável em um lugar só.

5. A memória do projeto: dois índices, duas perguntas

Como todo agente começa sem memória do que houve antes, o sistema entrega o conhecimento pronto, em camadas — e distingue duas perguntas que costumam ser confundidas quando se fala em memória para agentes.

A pergunta	O que responde	Propriedades
“O que existe e como se chama?”	Índice gerado por script: árvore de arquivos com a assinatura de cada função e uma linha de propósito	Exato, completo, custo zero, sempre atualizado; vai inteiro no prompt
“Isto já foi resolvido aqui, e o que foi decidido?”	Índice por significado sobre a história do projeto — decisões com o motivo, achados de revisão, tarefas concluídas	Aproximado, sob demanda, só os melhores trechos, com a fonte citada

Dois detalhes que a distinção esclarece. O material indexado por significado **não é o código** — disso o índice gerado já dá conta, de graça e com exatidão; é a história do projeto. E a busca é híbrida porque o vetor perde nome próprio, enquanto a busca textual perde sinônimo: uma cobre o buraco da outra. A documentação inclui ainda a regra de quando **não** usar o índice por significado — ele não substitui o índice exato nem responde o que uma consulta ao banco responde melhor. Fora disso, é custo com cara de sofisticação.

6. O que é herdado, e citado

Há sobreposição real, e ela deve ser declarada sem eufemismo:

Mecanismo	Origem a citar
O termo “fábrica de software” e a metáfora da linha de produção	CUSUMANO, 1991 — anterior aos modelos de linguagem
Papéis especializados de agentes produzindo software	ChatDev e MetaGPT, 2023
Separação entre quem escreve e quem revisa	ChatDev — o revisor aponta, o programador corrige
Limite declarado de iteração	ChatDev — dez rodadas ou duas modificações sem mudança
Portão de qualidade, commit por unidade, modelo por papel, estimativa de custo	softwarefabrik.io
Agente como processo supervisionado em OTP	Cortex, Sagents, SwarmEx, Synapse — prática corrente em Elixir

São itens a citar, não a reivindicar.

7. Convergência independente, e não plágio

Três argumentos, em ordem de força:

- **As soluções divergem onde o problema é o mesmo.** A autoaprovação é atacada pelos três sistemas. O comercial resolve pondo um humano no portão, ao custo da autonomia; os acadêmicos, por convenção do protocolo de diálogo; este, por ausência da ferramenta no catálogo do papel. Divergência sob problema comum é o oposto de cópia.
- **O regime é diferente, e os mecanismos acompanham.** Contexto por papel, começo reaproveitado e estado que sobrevive à sessão não são melhorias sobre os sistemas de sessão única: são exigências de um regime que eles não enfrentam.
- **A cronologia mostra derivação, não transcrição.** Os conceitos aparecem no repositório em sequência, cada um depois do incidente que o motivou, ao longo de seis semanas.

POSICIONAMENTO SUGERIDO O trabalho deve ser apresentado em diálogo com a literatura, e não em oposição a ela: adota o diagnóstico e o vocabulário já estabelecidos, herda mecanismos citados nominalmente, e investiga uma pergunta que os antecessores não fazem — se a separação entre construir e aprovar se sustenta sem operador humano no laço, num regime de semanas, e quanto isso custa. É contribuição incremental declarada, que é o que se espera de um trabalho de conclusão.

8. Evidência

O repositório é público, com 205 commits entre 18 de julho e 28 de agosto de 2026. Os mecanismos centrais têm commit e data:

Commit	Data	O que registra
8215dbf	01/08	Índice denso do projeto no lugar da varredura de arquivos
9d43b61	01/08	Doutrina de custo: o modelo, a medição e as cinco alavancas
1a41ca8	02/08	Montador de contexto por papel e teto de custo com parada limpa
4c559ef	02/08	Começo reaproveitável e critérios de aceite executáveis
91559b8	02/08	O laço de orquestração vira código testado
8017c41	02/08	O pipeline decide sozinho, sem portão de julgamento humano
e0bffee	23/08	Fechamento do portão nos dois caminhos de saída do construtor

Medições que contrariaram a própria hipótese

O sinal de que o método é honesto não é a medição que confirma o que já se supunha, e sim a que obriga a mudar de rumo:

- **174f0a0 (31/07)** — **previsões escritas antes da rodada**, para confrontá-las com o resultado depois.
- **696762e (31/07)** — **um ganho de 12% foi remedido e identificado como ruído**, e a afirmação foi retirada.
- **dea9e9c (31/07)** — **a contagem de saída estava com vezes subestimada**; o instrumento foi corrigido antes de qualquer conclusão.
- **e8974c5 (21/08)** — **hipótese de gargalo avaliada e descartada**: supunha-se que o portão de verificação limitaria a vazão; medido sobre quarenta e três rodadas, houve zero segundo de espera, e a otimização já desenhada foi abandonada.

Em operação real, a primeira versão entregou 86 tarefas concluídas de 89, em três projetos de domínios distintos — um jogo de tabuleiro em versão web multijogador em tempo real; um serviço permanente embarcado no computador de placa única que controla uma impressora 3D; e uma plataforma de dados com camada de análise por modelos de linguagem. São 139 execuções gravadas e preservadas no repositório.

SOBRE A EXTENSÃO A ARTEFATOS NÃO-SOFTWARE O sistema também opera fora de software, com uma escada de prova que obriga o verificador a declarar se o critério foi executado, inspecionado ou julgado. É extensão do mesmo princípio — se a prova não vem de graça, ela tem de ser declarada — e vale registro, mas não é a contribuição central deste trabalho.

9. Encaminhamento

- **Declarar o regime na primeira página do trabalho.** É ele que torna a comparação honesta e que explica por que metade dos mecanismos existe.
- **Escrever a seção de trabalhos relacionados e citar antes de ser questionado:** ChatDev e MetaGPT; softwarefabrik.io e Factory.ai; os frameworks de agentes em Elixir/OTP; e Cusumano pelo termo.
- **Enunciar a contribuição pela pergunta, não pelo artefato:** não “construí uma fábrica de software multi-agente”, e sim “investiguei se a separação entre construir e aprovar se sustenta sem operador humano no laço, num regime de semanas, e medi o que isso custa”.

Referências

- CUSUMANO, M. Japan's Software Factories. Oxford University Press, 1991.
- QIAN, C. et al. ChatDev: Communicative Agents for Software Development. arXiv:2307.07924, 2023.
- HONG, S. et al. MetaGPT: Meta Programming for a Multi-Agent Collaborative Framework, 2023.

- JANDA, M. Agentic Software Factory. softwarefabrik.io, v0.30.0, 2026.
- Factory.ai. Factory 2.0: from coding agents to software factories.
- Cortex (github.com/itsHabib/cortex); Sagents (github.com/sagents-ai/sagents).